Assignment 1

Fernando Melo Filho
fmelofilho2950@floridapoly.edu

CAP 5410 — Computer Vision
Assignment 1

Instructor: Dr. Muhammad Abid

Brightness and Contrast Editing

Department of Computer Science
Florida Polytechnic University, Florida, USA

September 2026

CONTENTS

CONTENTS ii

LIST OF FIGURES iv

LIST OF TABLES v

1 INTRODUCTION 1

1.1 OBJECTIVES 1

1.2 BACKGROUND 1

1.3 REPORT ORGANIZATION 2

2 METHODOLOGY AND IMPLEMENTATION 3

2.1 DEVELOPMENT ENVIRONMENT 3

2.2 APPLICATION ARCHITECTURE 3

2.3 TASK 1 — LOADING AND DISPLAYING THE SOURCE VIDEO 4

2.4 TASK 2 — HISTOGRAM OF THE SOURCE VIDEO 5

2.5 TASK 3 — PREVIEW (ADJUSTED) VIDEO 6

2.6 TASK 4 — HISTOGRAM OF THE PREVIEW VIDEO 6

2.7 TASK 5 — CONTINUOUS LOOPING PLAYBACK 7

2.8 TASK 6 — BRIGHTNESS AND CONTRAST SCROLL BARS 8

2.9 TASK 7 — SAVE FUNCTIONALITY 9

2.10 TASK 8 — PERSISTING SETTINGS ACROSS RESTARTS 10

2.11 ADDITIONAL FEATURES 11

2.12 REQUIREMENTS TRACEABILITY 11

3 RESULTS 13

3.1 INPUT VIDEO 13

3.2 MAIN WINDOW AT DEFAULT SETTINGS 14

3.3 EFFECT OF THE BRIGHTNESS CONTROL 14

3.4 EFFECT OF THE CONTRAST CONTROL 15

3.5 CONTINUOUS PLAYBACK 15
3.6 SAVING THE ADJUSTED VIDEO 16
3.7 SETTINGS RESTORED AFTER RELOAD 16

4 DISCUSSION AND CONCLUSION 17
4.1 DISCUSSION 17
4.2 CHALLENGES AND LIMITATIONS 17
4.3 CONCLUSION 17

REFERENCES 18

APPENDIX A SOURCE CODE 19

LIST OF FIGURES

Figure 2.1 Application architecture. 4

Figure 3.1 Main window at default settings. 14
Figure 3.2 Effect of the brightness control. 14
Figure 3.3 Effect of the contrast control. 15
Figure 3.4 Automatic restart at end of stream. 15
Figure 3.5 Save operation. 16
Figure 3.6 Settings restored on reload. 16

LIST OF TABLES

Table 2.1 Software environment. 3
Table 2.2 Key methods of the `VideoEditor` class. 4
Table 2.3 Adjustment controls. 8
Table 2.4 Additional features. 12
Table 2.5 Requirements traceability. 12

Table 3.1 Input video properties. 13

1 INTRODUCTION

1.1 OBJECTIVES

The assignment requires implementing a program that satisfies the following eight tasks:

1. Load and display the source video.
2. Display the histogram of the source video.
3. Display the same video as an adjustable preview.
4. Display the histogram of the preview video.
5. Loop both videos continuously, so that each restarts from its first frame as soon as it ends.
6. Provide brightness and contrast scroll bars.
7. Provide a save function, triggered by a button, a key, or a key combination.
8. Persist the adjusted settings so that, when the program is reloaded, both windows reopen with the previous settings already applied.

1.2 BACKGROUND

Image processing can be described as a function that takes one or more images as input and produces an image as output. Once an image is loaded, its pixel values can be manipulated in a variety of ways. These manipulations fall into two broad categories: point operators and neighborhood operators (Szeliski, 2022). In a point operator, the value of an output pixel depends only on the value of the corresponding input pixel; in a neighborhood operator, the output pixel also depends on the pixels surrounding it.

Two of the most common point operations are multiplication and addition by a constant, combined into a single expression:

$$g(x) = \alpha \cdot f(x) + \beta \tag{1.1}$$

where f is the input image and g is the output image. Applied to every pixel of a two-dimensional image (see Equation 1.2), the same operation is written as

$$g(x, y) = \alpha \cdot f(x, y) + \beta \tag{1.2}$$

The parameter α is called the *gain* and controls contrast, while β is called the *bias* and controls brightness. A dark image has many low-intensity pixels, so its histogram shows a peak toward the left (low) end. Adding a constant bias shifts every pixel by the same amount, which translates the whole histogram to the right for $\beta > 0$ or to the left for $\beta < 0$. Because pixel intensities are stored as 8-bit values, any result above 255 is clipped to 255 and any result below 0 is clipped to 0.

1.3 REPORT ORGANIZATION

The remainder of this report is organized as follows. Chapter 2 describes the development environment, the application architecture, and how each of the eight tasks was implemented. Chapter 3 presents sample outputs of the program for each task. Chapter 4 discusses the results, notes the limitations encountered, and concludes the report.

2 METHODOLOGY AND IMPLEMENTATION

2.1 DEVELOPMENT ENVIRONMENT

Table 2.1 lists the language and libraries I used to build and run the program, with their versions. Video capture, writing, and histogram computation all go through OpenCV (Bradski, 2000); array operations use NumPy (Harris et al., 2020); frames are converted for display with Pillow (Pillow Contributors, 2026); and the interface is built with Tkinter (Python Software Foundation, 2026).

Table 2.1. Software environment used to develop and run the program.

Component	Version	Purpose
Python	3.13.5	Programming language used to implement the application
OpenCV (opencv-python)	5.0.0.93	Video capture and writing, frame processing, and histogram computation
NumPy	2.5.2	Array operations
Pillow (PIL)	12.3.0	Converts OpenCV BGR frames into images that Tkinter can display
Tkinter	bundled with Python (Tcl/Tk)	GUI toolkit: windows, scroll bars, buttons and labels

2.2 APPLICATION ARCHITECTURE

I implemented the whole application as a single class, `VideoEditor` (Listing 2.1), and grouped its methods by responsibility, as shown in Table 2.2.

```
1 class VideoEditor:
2     def __init__(self, root, path=SOURCE):
3         self.root = root
4         self.path = path
```

Listing 2.1. The `VideoEditor` class and constructor signature (`main.py`).

Table 2.2. Key methods of the `VideoEditor` class and their responsibility.

Method	Responsibility
`__init__`	Opens the source video, restores persisted settings, builds the UI, and starts the main loop
`_build_ui` / `_video_column`	Construct the video panels, histogram panels, sliders, and buttons
`_process`	Applies the brightness/contrast point operation (Equation 1.2) to a frame
`_hist_image`	Renders a per-channel histogram into a small BGR canvas
`_show`	Converts a BGR frame into a Tkinter-displayable image
`_read_looping` / `_tick`	Timer-driven main loop: reads frames, loops at end of stream, and refreshes all four panels
`_load_settings` / `_store_settings`	Read and write `settings.json` to persist slider values between runs
`_save` / `_save_worker` / `_poll_save`	Prompt for an output file, export it on a background thread, and report progress
`_toggle_pause` / `_reset` / `close`	Pause/resume playback, reset the sliders, and release resources on exit

Figure 2.1. <Caption: block diagram of the processing pipeline and window layout.>

2.3 TASK 1 — LOADING AND DISPLAYING THE SOURCE VIDEO

I open the source file once, in the constructor, with `cv2.VideoCapture`; if the file cannot be opened, I show an error dialog and close the window (Listing 2.2).

```
        self.cap = cv.VideoCapture(path)
        if not self.cap.isOpened():
            messagebox.showerror("Error", f"Could not open '{path}'")
            root.destroy()
            return

        src_w = int(self.cap.get(cv.CAP_PROP_FRAME_WIDTH))
        src_h = int(self.cap.get(cv.CAP_PROP_FRAME_HEIGHT))
        self.disp_w = DISPLAY_W
```

```
        self.disp_h = int(src_h * DISPLAY_W / src_w)
```

Listing 2.2. Opening the source video and computing the display size (`main.py`).

Every decoded frame is a NumPy array in OpenCV's BGR channel order, so to display it inside a Tkinter label I convert it to RGB and wrap it in a `PIL.ImageTk.PhotoImage` (Pillow Contributors, 2026) (Listing 2.3). I keep a reference to the image in `self._refs`, since Tkinter does not hold its own reference and would otherwise garbage-collect it and leave the panel blank.

```
    def _show(self, label, bgr, key):
        rgb = cv.cvtColor(bgr, cv.COLOR_BGR2RGB)
        img = ImageTk.PhotoImage(Image.fromarray(rgb))
        label.configure(image=img)
        self._refs[key] = img           # without this the panel goes blank
```

Listing 2.3. Converting a BGR frame into a Tkinter-displayable image (`main.py`).

2.4 TASK 2 — HISTOGRAM OF THE SOURCE VIDEO

For each of the three color channels, I use `cv2.calcHist` (Bradski, 2000) to bin the frame into 256 bins over the range $[0, 256)$, normalize the result to the height of the histogram canvas, and draw it as a polyline (Listing 2.4).

```
    def _hist_image(self, frame, w, h):
        """Draw a 3-channel histogram into a BGR image."""
        canvas = np.full((h, w, 3), 28, np.uint8)
        colors = ((255, 110, 110), (110, 255, 110), (110, 110, 255))  # B, G, R
        xs = np.linspace(0, w - 1, 256).astype(np.int32)

        for ch, color in enumerate(colors):
            hist = cv.calcHist([frame], [ch], None, [256], [0, 256]).flatten()
            cv.normalize(hist, hist, 0, h - 6, cv.NORM_MINMAX)
            ys = (h - 3 - hist).astype(np.int32)
            pts = np.stack([xs, ys], axis=1).reshape(-1, 1, 2)
            cv.polylines(canvas, [pts], False, color, 1, cv.LINE_AA)

        return canvas
```

Listing 2.4. Computing and drawing the per-channel histogram (`main.py`).

I call this routine once per tick for the original frame, so the histogram panel refreshes at the same rate as the video (Listing 2.5).

```
            self._show(self.orig_hist,
                       self._hist_image(left, self.disp_w, HIST_H), "oh")
```

Listing 2.5. Refreshing the original video's histogram panel every frame (`main.py`).

2.5 TASK 3 — PREVIEW (ADJUSTED) VIDEO

Both panels come from the same decoded frame, so they can never fall out of sync: I get the right-hand ("Edited") panel by applying a brightness/contrast point operation to a copy of that frame with `cv2.convertScaleAbs` (Listing 2.6), which computes the linear mapping and clips the result to $[0, 255]$ in one call.

```
    def _process(self, frame):
        """The one place editing happens. Pure function of (frame, params)
            .

        g(x) = alpha*(f(x) - 127.5) + 127.5 + beta

        Contrast (alpha) scales around mid-gray (127.5) instead of zero,
            so
        it stretches/compresses the tonal range without also shifting the
        overall brightness. Brightness (beta) is the only additive term,
            so
        it's the only control that uniformly lightens/darkens the frame.
        Rewritten as alpha*f(x) + beta' so cv.convertScaleAbs can still do
        the scaling and clipping in one call.
        """
        alpha = self.contrast.get()
        beta = self.brightness.get()
        beta_centered = beta + 127.5 * (1.0 - alpha)
        return cv.convertScaleAbs(frame, alpha=alpha, beta=beta_centered)
```

Listing 2.6. The brightness/contrast point operation, centered on mid-gray, applied to a frame (`main.py`).

```
            small = cv.resize(self.current, (self.disp_w, self.disp_h))
            edited = self._process(small)
```

Listing 2.7. Building the adjusted (preview) frame once per tick (`main.py`).

2.6 TASK 4 — HISTOGRAM OF THE PREVIEW VIDEO

I reuse the same `_hist_image` routine from Section 2.4 on the adjusted frame, so both histograms are always computed identically and only differ because of the brightness/contrast operation itself (Listing 2.8).

```
            self._show(self.edit_hist,
                       self._hist_image(edited, self.disp_w, HIST_H), "eh"
                           )
```

Listing 2.8. Refreshing the preview video's histogram panel every frame (`main.py`).

2.7 TASK 5 — CONTINUOUS LOOPING PLAYBACK

`cv2.VideoCapture.read` returns `False` once the last frame has been consumed; when that happens, `_read_looping` rewinds the capture to frame 0 with `CAP_PROP_POS_FRAMES` and reads again, so playback never stops (Listing 2.9).

```
    @staticmethod
    def _read_looping(cap):
        """Next frame, rewinding to frame 1 at end of file."""
        ret, frame = cap.read()
        if not ret:
            cap.set(cv.CAP_PROP_POS_FRAMES, 0)
            ret, frame = cap.read()
        return frame if ret else None
```

Listing 2.9. Reading the next frame and rewinding to frame 0 at end of stream (`main.py`).

I drive playback with `_tick`, a callback that Tkinter's event loop re-invokes every `FRAME_DELAY` (30 ms, $\approx$ 33 fps) through `root.after`. Each call reads one frame from each capture, builds the four panels described in Sections 2.3–2.6, and reschedules itself, which is what keeps both videos playing continuously (Listing 2.10).

```
    def _tick(self):
        if not self.paused:
            frame = self._read_looping(self.cap)
            if frame is not None:
                self.current = frame

            if self.saved_cap is not None:
                frame = self._read_looping(self.saved_cap)
                if frame is not None:
                    self.saved_frame = frame

        # left panel: the exported file when we have one, else the source
        left_src = self.saved_frame if self._using_saved() else self.
            current

        if self.current is not None:
            if left_src is None:
                left_src = self.current
            left = cv.resize(left_src, (self.disp_w, self.disp_h))
            small = cv.resize(self.current, (self.disp_w, self.disp_h))
            edited = self._process(small)

            self._show(self.orig_video, left, "ov")
            self._show(self.edit_video, edited, "ev")
            self._show(self.orig_hist,
                       self._hist_image(left, self.disp_w, HIST_H), "oh")
            self._show(self.edit_hist,
                       self._hist_image(edited, self.disp_w, HIST_H), "eh"
                           )

        self.root.after(FRAME_DELAY, self._tick)
```

Listing 2.10. The timer-driven main loop that keeps both videos playing continuously (`main.py`).

2.8 TASK 6 — BRIGHTNESS AND CONTRAST SCROLL BARS

I built the two scroll bars as `ttk.Scale` (Python Software Foundation, 2026) widgets bound to the `tk.DoubleVar` instances `self.brightness` and `self.contrast` (Listing 2.11). Because `_process` (Listing 2.6) reads these variables directly every tick, moving a slider takes effect on the very next rendered frame, with no extra event wiring needed.

```
        ttk.Label(right, text="Brightness").pack(anchor="w")
        ttk.Scale(right, from_=-100, to=100, orient="horizontal",
                  variable=self.brightness, length=180,
                  command=lambda v: self._sync_labels()).pack(fill="x")
        self.b_label = ttk.Label(right, text="0")
        self.b_label.pack(anchor="e")

        ttk.Label(right, text="Contrast").pack(anchor="w", pady=(12, 0))
        ttk.Scale(right, from_=0.1, to=3.0, orient="horizontal",
                  variable=self.contrast, length=180,
                  command=lambda v: self._sync_labels()).pack(fill="x")
        self.c_label = ttk.Label(right, text="1.00")
        self.c_label.pack(anchor="e")
```

Listing 2.11. Brightness and contrast scroll bars (`main.py`).

Table 2.3 lists the range and default of each control, corresponding to α and β in Equation 1.2.

Table 2.3. Adjustment controls, their ranges and their effect on the image.

Control	Symbol	Range (default)	Effect
Brightness	β	-100 to 100 (0)	Additive offset: shifts every pixel intensity up or down, translating the histogram left or right; clipped to $[0, 255]$
Contrast	α	0.1 to 3.0 (1.0)	Multiplicative gain pivoted on mid-gray (127.5): $\alpha > 1$ stretches the histogram outward from the center (higher contrast), $\alpha < 1$ compresses it toward the center (lower contrast), without shifting the overall brightness; also clipped to $[0, 255]$

2.9 TASK 7 — SAVE FUNCTIONALITY

I trigger saving either from the "Save Video..." button or with `Ctrl+S` (both bound in the constructor, Listing 2.1). I snapshot the current α and β values at that moment, so later slider movements do not affect an export already in progress (Listing 2.12).

```
    def _save(self):
        if self.saving:
            return
        out_path = filedialog.asksaveasfilename(
            defaultextension=".avi",
            filetypes=[("AVI", "*.avi"), ("MP4", "*.mp4")],
            initialfile="edited.avi")
        if not out_path:
            return

        self.saving = True
        self.save_error = None
        self.save_progress = 0.0
        self._pending_out = out_path
        self.save_btn.state(["disabled"])
        self.status.configure(text="Saving...")

        # snapshot the settings so later slider moves don't affect this
            export
        args = (out_path, self.contrast.get(), self.brightness.get())
        threading.Thread(target=self._save_worker, args=args, daemon=True)
            .start()
        self._poll_save()
```

Listing 2.12. Triggering the save dialog and starting the export on a background thread (`main.py`).

The export re-opens the source file at full resolution, instead of reusing the down-scaled preview frames, and runs on a background thread so both video panels keep updating while it works (Listings 2.13 and 2.14).

```
    def _save_worker(self, out_path, alpha, beta):
        """Runs off the UI thread. Re-reads the source at full resolution.
            """
        cap = writer = None
        try:
            cap = cv.VideoCapture(self.path)
            fps = cap.get(cv.CAP_PROP_FPS) or 25.0
            w = int(cap.get(cv.CAP_PROP_FRAME_WIDTH))
            h = int(cap.get(cv.CAP_PROP_FRAME_HEIGHT))
            total = int(cap.get(cv.CAP_PROP_FRAME_COUNT)) or 1

            ext = os.path.splitext(out_path)[1].lower()
            fourcc = cv.VideoWriter_fourcc(*("mp4v" if ext == ".mp4" else
                "XVID"))
            writer = cv.VideoWriter(out_path, fourcc, fps, (w, h))
            if not writer.isOpened():
                raise IOError("VideoWriter could not be opened")
```

Listing 2.13. Re-opening the source at full resolution and preparing the output writer (`main.py`, excerpt).

```
            n = 0
            while True:
                ret, frame = cap.read()
                if not ret:
                    break
                writer.write(cv.convertScaleAbs(frame, alpha=alpha, beta=
                    beta))
                n += 1
                self.save_progress = min(100.0, 100.0 * n / total)
```

Listing 2.14. Applying the settings to each frame and tracking export progress (`main.py`, excerpt).

`_poll_save` (Listing 2.15) is the only place I touch Tkinter widgets while the export runs; the worker thread only ever writes to plain instance attributes (`self.save_progress`, `self.save_error`), which keeps me from touching Tkinter from the wrong thread.

```
    def _poll_save(self):
        """UI-thread poller: never touch widgets from the worker thread."""
        self.progress["value"] = self.save_progress
        if self.saving:
            self.root.after(100, self._poll_save)
        else:
            self.progress["value"] = 0 if self.save_error else 100
            self.save_btn.state(["!disabled"])
            if self.save_error:
                self.status.configure(text="Save failed", foreground="#b00")
                messagebox.showerror("Save failed", self.save_error)
            else:
                # the export becomes the left panel's source from now on,
                # and survives a restart through settings.json
                self.saved_path = self._pending_out
                self._open_saved()
                self.cap.set(cv.CAP_PROP_POS_FRAMES, 0)
                self._store_settings()
                self.status.configure(text="Saved", foreground="#070")
```

Listing 2.15. Polling the export status from the UI thread and reporting the result (`main.py`).

2.10 TASK 8 — PERSISTING SETTINGS ACROSS RESTARTS

I store brightness, contrast, and the path of the last exported file in a JSON file, `settings.json`, next to the script. `_load_settings` reads it back when the program starts, and `_store_settings` writes it whenever the program closes and again right after a successful save, so both windows reopen with the same brightness, contrast, and exported-video source as before.

```
    def _load_settings(self):
        """Read settings.json. A missing or broken file is not an error."""
        try:
            with open(SETTINGS, "r", encoding="utf-8") as fh:
                data = json.load(fh)
            return data if isinstance(data, dict) else {}
        except (OSError, ValueError):
            return {}

    def _store_settings(self):
        """Write the current settings so the next launch reproduces them."""
        data = {
            "brightness": round(self.brightness.get(), 2),
            "contrast": round(self.contrast.get(), 3),
            "saved_path": self.saved_path,
            "show_saved": bool(self.show_saved.get()),
        }
        try:
            with open(SETTINGS, "w", encoding="utf-8") as fh:
                json.dump(data, fh, indent=2)
        except OSError:
            pass                        # never let a settings write break
                the app
```

Listing 2.16. Reading and writing `settings.json` (`main.py`).

```
    def close(self):
        self._store_settings()
        self.cap.release()
        if self.saved_cap is not None:
            self.saved_cap.release()
```

Listing 2.17. Settings are written back when the window closes (`main.py`).

2.11 ADDITIONAL FEATURES

Table 2.4 lists a few features I added beyond the eight required tasks.

2.12 REQUIREMENTS TRACEABILITY

Table 2.5 maps each of the eight required tasks to the function(s) that implement it and to the figure in Chapter 3 that demonstrates it.

Feature	Implemented in	Trigger
Pause / resume playback	`_toggle_pause`	“Pause” button or spacebar
Reset sliders to defaults	`_reset`	“Reset” button
Save keyboard shortcut	`_save`	`Ctrl+S`
Export progress bar	`_poll_save`	Updates automatically while saving
Toggle left panel between the original source and the last exported video	`_toggle_saved`	“Left panel: exported video” checkbox

Table 2.4. Features implemented beyond the eight required tasks.

Table 2.5. Mapping of each assignment requirement to its implementation and to the figure that demonstrates it.

#	Requirement	Implemented in	Evidence
1	Load video on screen	`__init__`, `_show`	Fig. 3.1
2	Histogram of that video	`_hist_image`	Fig. 3.1
3	Same video as preview	`_process`	Fig. 3.1
4	Histogram of the preview	`_hist_image`	Fig. 3.1
5	Continuous looping	`_read_looping`, `_tick`	Fig. 3.4
6	Brightness / contrast bars	`_build_ui`	Figs. 3.2, 3.3
7	Save functionality	`_save`, `_save_worker`	Fig. 3.5
8	Settings restored on reload	`_load_settings`, `_store_settings`	Fig. 3.6

3 RESULTS

3.1 INPUT VIDEO

Table 3.1. Properties of the input video used for all results in this chapter.

Property	Value
File name	`barriers.avi`
Resolution (W $\times$ H)	640×480
Frame rate	50 fps
Frame count	399
Duration	≈ 8.0 s
Codec / container	FMP4 / AVI

3.2 MAIN WINDOW AT DEFAULT SETTINGS

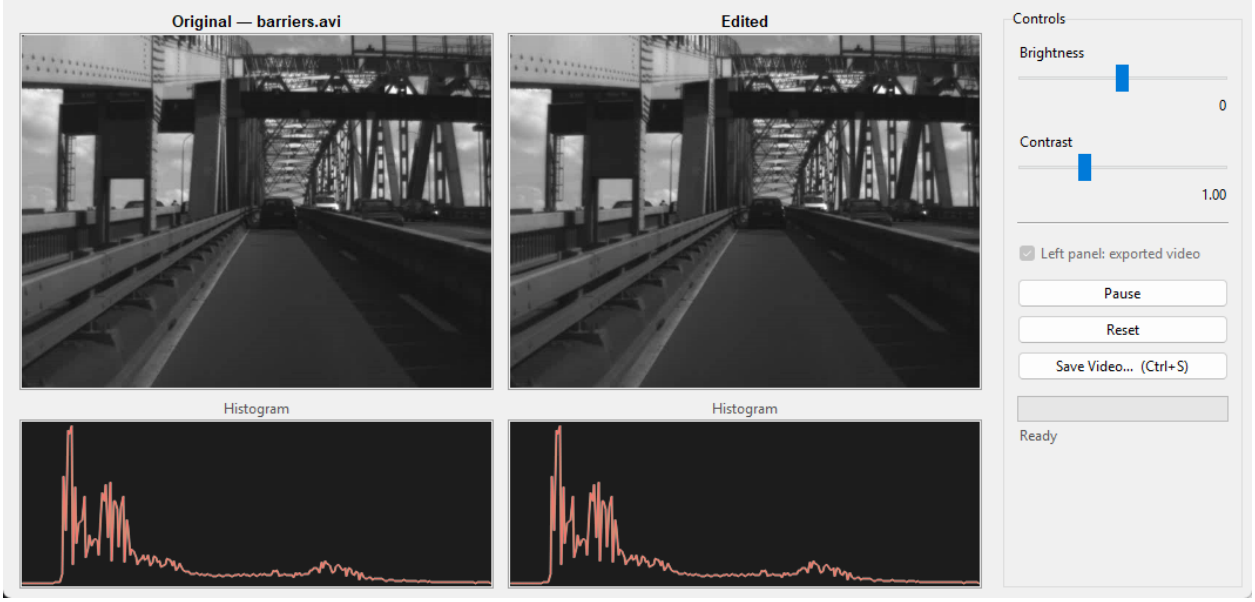
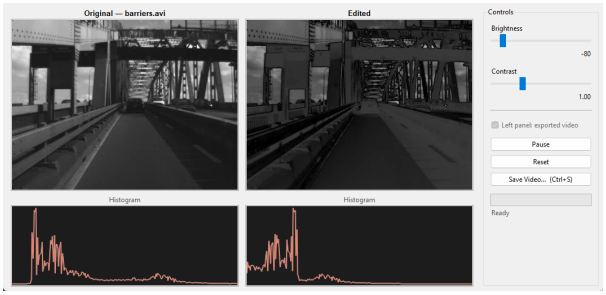
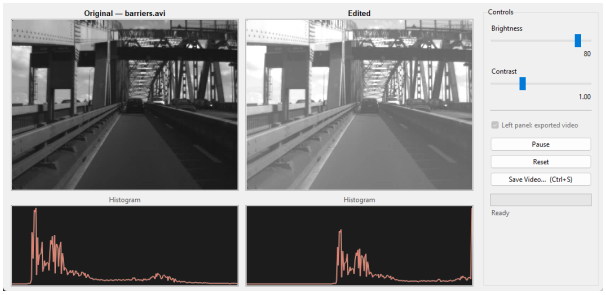


Figure 3.1. Full application window at the default settings ($\beta = 0$, $\alpha = 1.0$): the original video (left) and the preview video (right) with their respective histograms underneath, plus the control panel.

3.3 EFFECT OF THE BRIGHTNESS CONTROL



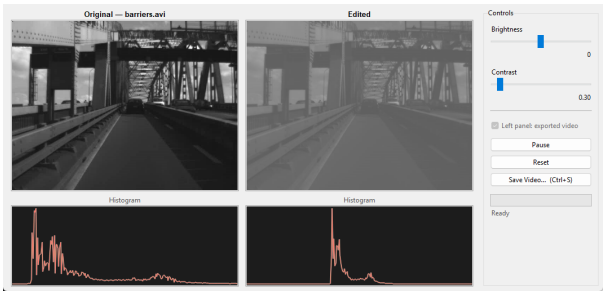
(a) $\beta = -80$



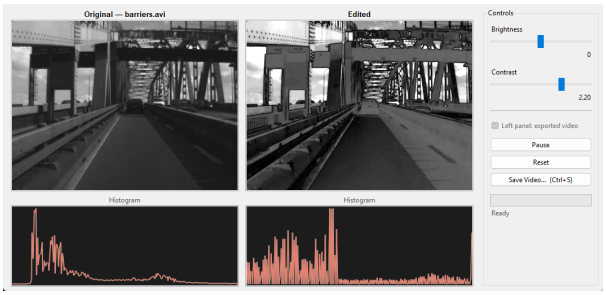
(b) $\beta = +80$

Figure 3.2. Preview video and its histogram at two brightness settings ($\alpha = 1.0$ fixed in both), compared against the unmodified original on the left of each panel. The histogram translates left for negative β and right for positive β, exactly as described in Table 2.3.

3.4 EFFECT OF THE CONTRAST CONTROL



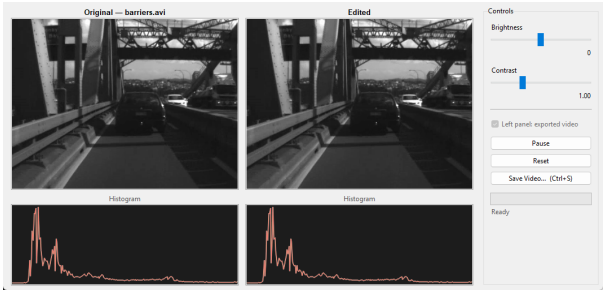
(a) $\alpha = 0.3$



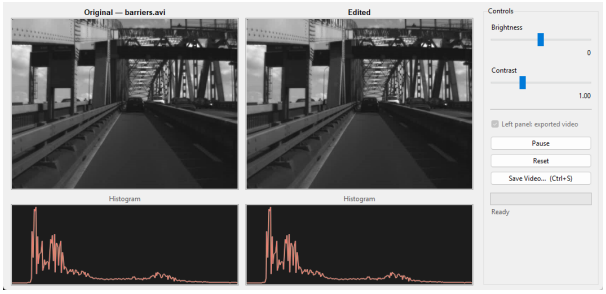
(b) $\alpha = 2.2$

Figure 3.3. Preview video and its histogram at two contrast settings ($\beta = 0$ fixed in both), compared against the unmodified original on the left of each panel. At $\alpha = 0.3$ the histogram compresses toward mid-gray; at $\alpha = 2.2$ it spreads out and several bins clip at 0 and 255.

3.5 CONTINUOUS PLAYBACK



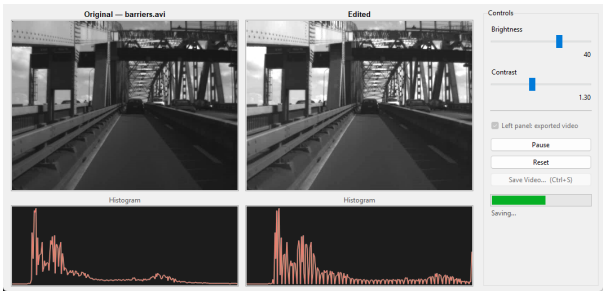
(a) Frame 399 (final frame)



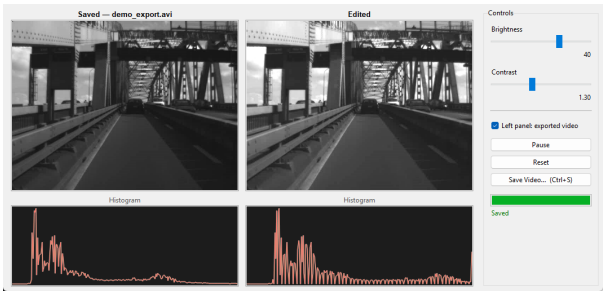
(b) Frame 1, immediately after the automatic restart

Figure 3.4. Playback wrapping from the final frame of `barriers.avi` back to frame 1 without any user action, exercising the same end-of-stream/rewind path (`_read_looping`, Listing 2.9) shown in Section 2.7.

3.6 SAVING THE ADJUSTED VIDEO



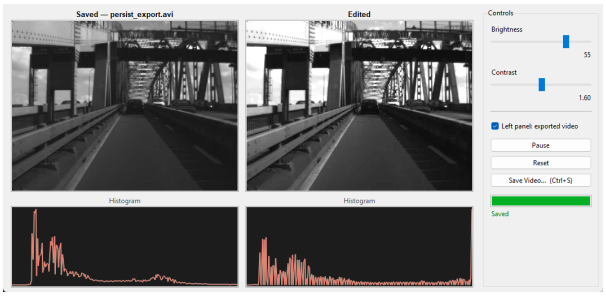
(a) Export in progress



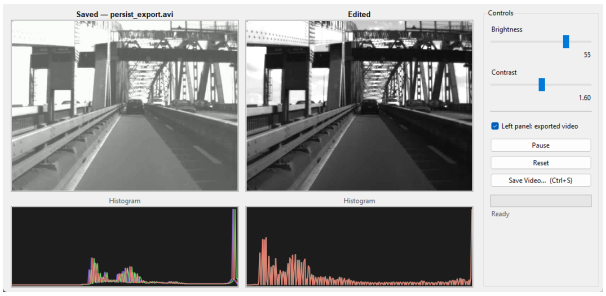
(b) Export complete

Figure 3.5. Saving with $\beta = 40$, $\alpha = 1.3$: (a) the "Save Video…" button disabled and the progress bar advancing while `_save_worker` re-encodes the source on a background thread; (b) after completion, the status reads "Saved" and the left panel automatically switches to play back the exported file (caption updates to "Saved — demo_export.avi").

3.7 SETTINGS RESTORED AFTER RELOAD



(a) Before closing the program



(b) After restarting the program

Figure 3.6. Brightness ($\beta = 55$), contrast ($\alpha = 1.6$), and the exported-video checkbox are all identical before closing (a) and immediately after a fresh launch (b), confirming that `settings.json` (Listing 2.16) round-trips correctly.

4 DISCUSSION AND CONCLUSION

4.1 DISCUSSION

As a software engineer, I like to split every project into a backend part and a frontend part; separating them from the start lets me reason about the two sets of technologies independently. By the backend I mean the calculations that change the image's settings — vector multiplication and so on. To actually see the result, rather than just debugging from the console, a frontend is needed. After some research, Tkinter was chosen because it is very simple to use: for now the interface only needs to be functional, and Tkinter gets the job done while keeping the code simple.

4.2 CHALLENGES AND LIMITATIONS

The main challenge was finding and organizing the functions needed to perform the operations. It ended up being a matter of identifying the right OpenCV (`cv2`) function for each task: `cv2.VideoCapture` and `cv2.VideoWriter` to read and write the video files, `cv2.calcHist` to build the histograms, `cv2.convertScaleAbs` to apply the brightness/contrast point operation with built-in clipping to $[0, 255]$, and `cv2.resize`/`cv2.cvtColor` to prepare each frame for display. After that, the next challenge was connecting these results to the UI. OpenCV frames are NumPy arrays in BGR channel order, while Tkinter widgets can only display `PhotoImage` objects, so each frame had to be converted to RGB and wrapped with `PIL.ImageTk.PhotoImage` before it could be shown on a `Label` — and a reference to that image had to be kept alive manually (Section 2.3), since Tkinter does not hold its own reference and would otherwise garbage-collect the image and leave the panel blank.

4.3 CONCLUSION

Brightness and contrast are point operations: they depend only on the input pixel to calculate the output. In this assignment, we showed how this works by adding sliders that change the video's brightness and contrast in real time, and observed how the histogram changes according to the values assigned to gain and bias.

REFERENCES

Bradski, G. 2000. “The OpenCV library.” *Dr. Dobb’s Journal of Software Tools*.

Harris, C. R. et al. 2020. “Array programming with NumPy.” *Nature*, 585: 357–362.

Pillow Contributors 2026. “Pillow documentation. Accessed September 2026.

Python Software Foundation 2026. “tkinter — Python interface to Tcl/Tk. Accessed September 2026.

Szeliski, R. 2022. *Computer Vision: Algorithms and Applications*. Springer, 2nd edition.

A SOURCE CODE

The complete, unmodified contents of `main.py` are listed below; this is the same file submitted alongside this report. The source code, the input video, and a README are also available at https://github.com/Fernando12filho/Brightness-Contrast-Display.

A.1 MAIN.PY

```
"""
Video Editor - side-by-side comparison with live histograms.

Layout
------
    +--------------------------+--------------------------+-----------------+
    |       ORIGINAL video     |       EDITED video       |                 |
    +--------------------------+--------------------------+   CONTROLS      |
    |    original histogram    |    edited histogram      |                 |
    +--------------------------+--------------------------+-----------------+

The sliders affect ONLY the right-hand video. The left stays untouched
so you always have a reference.

Settings (brightness, contrast, last exported file) are written to
settings.json, so relaunching the program restores the previous session:
the sliders come back where they were and the left panel plays the last
exported video, i.e. both windows open with the new settings applied.

Requires: opencv-python, pillow, numpy
    pip install opencv-python pillow numpy
"""

import json
import os
import threading
import tkinter as tk
from tkinter import ttk, filedialog, messagebox

import cv2 as cv
import numpy as np
```

```
from PIL import Image, ImageTk

SOURCE = "barriers.avi"
SETTINGS = "settings.json"
DISPLAY_W = 400         # medium-size preview, per panel
HIST_H = 140            # histogram strip height
FRAME_DELAY = 30        # ms between frames  (~33 fps)


class VideoEditor:
    def __init__(self, root, path=SOURCE):
        self.root = root
        self.path = path

        self.cap = cv.VideoCapture(path)
        if not self.cap.isOpened():
            messagebox.showerror("Error", f"Could not open `{path}`")
            root.destroy()
            return

        src_w = int(self.cap.get(cv.CAP_PROP_FRAME_WIDTH))
        src_h = int(self.cap.get(cv.CAP_PROP_FRAME_HEIGHT))
        self.disp_w = DISPLAY_W
        self.disp_h = int(src_h * DISPLAY_W / src_w)

        self.current = None           # most recent raw frame (full
            resolution)
        self.saved_frame = None       # most recent frame of the exported
            file
        self.paused = False
        self.saving = False
        self.save_progress = 0.0
        self.save_error = None
        self._pending_out = None

        # ---- restore settings from the last session ----
        cfg = self._load_settings()

        self.brightness = tk.DoubleVar(value=cfg.get("brightness", 0.0))
        self.contrast = tk.DoubleVar(value=cfg.get("contrast", 1.0))

        self.saved_path = cfg.get("saved_path") or None
        self.saved_cap = None                       # capture over the
            exported file
        self.show_saved = tk.BooleanVar(value=bool(cfg.get("show_saved",
            True)))

        # keeps PhotoImage objects alive; Tkinter does not hold its own
            ref
        self._refs = {}

        self._build_ui()
        self._open_saved()            # left panel picks up the exported
            video
```

```
        self._sync_labels()
        self.root.bind("<space>", lambda e: self._toggle_pause())
        self.root.bind("<Control-s>", lambda e: self._save())
        self.root.bind("<Control-S>", lambda e: self._save())
        self._tick()

    # ---- persistence ----

    def _load_settings(self):
        """Read settings.json. A missing or broken file is not an error."""
        try:
            with open(SETTINGS, "r", encoding="utf-8") as fh:
                data = json.load(fh)
            return data if isinstance(data, dict) else {}
        except (OSError, ValueError):
            return {}

    def _store_settings(self):
        """Write the current settings so the next launch reproduces them."""
        data = {
            "brightness": round(self.brightness.get(), 2),
            "contrast": round(self.contrast.get(), 3),
            "saved_path": self.saved_path,
            "show_saved": bool(self.show_saved.get()),
        }
        try:
            with open(SETTINGS, "w", encoding="utf-8") as fh:
                json.dump(data, fh, indent=2)
        except OSError:
            pass                        # never let a settings write break the app

    def _open_saved(self):
        """(Re)attach the capture that feeds the left-hand panel."""
        if self.saved_cap is not None:
            self.saved_cap.release()
            self.saved_cap = None
        self.saved_frame = None

        if self.saved_path and os.path.exists(self.saved_path):
            cap = cv.VideoCapture(self.saved_path)
            if cap.isOpened():
                self.saved_cap = cap
            else:
                cap.release()
                self.saved_path = None
        else:
            self.saved_path = None

        self._refresh_left_caption()

    def _using_saved(self):
```

```
        return self.saved_cap is not None and self.show_saved.get()

    def _refresh_left_caption(self):
        """The left column always states which file it is actually playing."""
        name = self.saved_path if self._using_saved() else self.path
        prefix = "Saved" if self._using_saved() else "Original"
        self.orig_caption.configure(text=f"{prefix} - {os.path.basename(name)}")
        self.saved_check.state(
            ["!disabled" if self.saved_cap is not None else "disabled"])

    def _toggle_saved(self):
        """Checkbox handler: rewind so the two panels stay frame-aligned."""
        if self.saved_cap is not None:
            self.saved_cap.set(cv.CAP_PROP_POS_FRAMES, 0)
        self.cap.set(cv.CAP_PROP_POS_FRAMES, 0)
        self.saved_frame = None
        self._refresh_left_caption()

    # ---- UI ----

    def _build_ui(self):
        self.root.title("Video Editor")

        outer = ttk.Frame(self.root, padding=8)
        outer.pack(fill="both", expand=True)

        # ---- LEFT: two video columns, histogram under each ----
        left = ttk.Frame(outer)
        left.grid(row=0, column=0, sticky="nw")

        (self.orig_caption, self.orig_video,
         self.orig_hist) = self._video_column(left, 0, "Original")
        (_, self.edit_video,
         self.edit_hist) = self._video_column(left, 1, "Edited")

        # ---- RIGHT: control panel ----
        right = ttk.LabelFrame(outer, text="Controls", padding=10)
        right.grid(row=0, column=1, sticky="nsew", padx=(12, 0))

        ttk.Label(right, text="Brightness").pack(anchor="w")
        ttk.Scale(right, from_=-100, to=100, orient="horizontal",
                  variable=self.brightness, length=180,
                  command=lambda v: self._sync_labels()).pack(fill="x")
        self.b_label = ttk.Label(right, text="0")
        self.b_label.pack(anchor="e")

        ttk.Label(right, text="Contrast").pack(anchor="w", pady=(12, 0))
        ttk.Scale(right, from_=0.1, to=3.0, orient="horizontal",
                  variable=self.contrast, length=180,
                  command=lambda v: self._sync_labels()).pack(fill="x")
        self.c_label = ttk.Label(right, text="1.00")
```

```
        self.c_label.pack(anchor="e")

        ttk.Separator(right, orient="horizontal").pack(fill="x", pady=14)

        self.saved_check = ttk.Checkbutton(
            right, text="Left panel: exported video",
            variable=self.show_saved, command=self._toggle_saved)
        self.saved_check.pack(anchor="w", pady=(0, 8))

        self.play_btn = ttk.Button(right, text="Pause", command=self._toggle_pause)
        self.play_btn.pack(fill="x", pady=3)

        ttk.Button(right, text="Reset", command=self._reset).pack(fill="x", pady=3)

        self.save_btn = ttk.Button(right, text="Save Video.. (Ctrl+S)",
                                   command=self._save)
        self.save_btn.pack(fill="x", pady=3)

        self.progress = ttk.Progressbar(right, mode="determinate", maximum=100)
        self.progress.pack(fill="x", pady=(10, 2))
        self.status = ttk.Label(right, text="Ready", foreground="#555")
        self.status.pack(anchor="w")

    def _video_column(self, parent, col, caption):
        """One column: caption, video panel, histogram panel underneath."""
        frame = ttk.Frame(parent)
        frame.grid(row=0, column=col, padx=6)

        # fixed width so swapping the caption cannot resize the window
        title = ttk.Label(frame, text=caption, width=30, anchor="center",
                          font=("TkDefaultFont", 10, "bold"))
        title.pack()

        video = ttk.Label(frame, borderwidth=1, relief="solid")
        video.pack()

        ttk.Label(frame, text="Histogram", foreground="#555").pack(pady=(6, 0))

        hist = ttk.Label(frame, borderwidth=1, relief="solid")
        hist.pack()

        return title, video, hist

    def _sync_labels(self):
        self.b_label.configure(text=f"{self.brightness.get():.0f}")
        self.c_label.configure(text=f"{self.contrast.get():.2f}")

    # ---- image pipeline ----

```

```
    def _process(self, frame):
        """The one place editing happens. Pure function of (frame, params)
           .

        g(x) = alpha*(f(x) - 127.5) + 127.5 + beta

        Contrast (alpha) scales around mid-gray (127.5) instead of zero,
           so
        it stretches/compresses the tonal range without also shifting the
        overall brightness. Brightness (beta) is the only additive term,
           so
        it's the only control that uniformly lightens/darkens the frame.
        Rewritten as alpha*f(x) + beta` so cv.convertScaleAbs can still do
        the scaling and clipping in one call.
        """
        alpha = self.contrast.get()
        beta = self.brightness.get()
        beta_centered = beta + 127.5 * (1.0 - alpha)
        return cv.convertScaleAbs(frame, alpha=alpha, beta=beta_centered)

    def _hist_image(self, frame, w, h):
        """Draw a 3-channel histogram into a BGR image."""
        canvas = np.full((h, w, 3), 28, np.uint8)
        colors = ((255, 110, 110), (110, 255, 110), (110, 110, 255))  # B,
            G, R
        xs = np.linspace(0, w - 1, 256).astype(np.int32)

        for ch, color in enumerate(colors):
            hist = cv.calcHist([frame], [ch], None, [256], [0, 256]).
               flatten()
            cv.normalize(hist, hist, 0, h - 6, cv.NORM_MINMAX)
            ys = (h - 3 - hist).astype(np.int32)
            pts = np.stack([xs, ys], axis=1).reshape(-1, 1, 2)
            cv.polylines(canvas, [pts], False, color, 1, cv.LINE_AA)

        return canvas

    def _show(self, label, bgr, key):
        rgb = cv.cvtColor(bgr, cv.COLOR_BGR2RGB)
        img = ImageTk.PhotoImage(Image.fromarray(rgb))
        label.configure(image=img)
        self._refs[key] = img          # without this the panel goes blank

    # ---- main loop ----

    @staticmethod
    def _read_looping(cap):
        """Next frame, rewinding to frame 1 at end of file."""
        ret, frame = cap.read()
        if not ret:
            cap.set(cv.CAP_PROP_POS_FRAMES, 0)
            ret, frame = cap.read()
        return frame if ret else None

```

```
    def _tick(self):
        if not self.paused:
            frame = self._read_looping(self.cap)
            if frame is not None:
                self.current = frame

            if self.saved_cap is not None:
                frame = self._read_looping(self.saved_cap)
                if frame is not None:
                    self.saved_frame = frame

        # left panel: the exported file when we have one, else the source
        left_src = self.saved_frame if self._using_saved() else self.
           current

        if self.current is not None:
            if left_src is None:
                left_src = self.current
            left = cv.resize(left_src, (self.disp_w, self.disp_h))
            small = cv.resize(self.current, (self.disp_w, self.disp_h))
            edited = self._process(small)

            self._show(self.orig_video, left, "ov")
            self._show(self.edit_video, edited, "ev")
            self._show(self.orig_hist,
                       self._hist_image(left, self.disp_w, HIST_H), "oh")
            self._show(self.edit_hist,
                       self._hist_image(edited, self.disp_w, HIST_H), "eh"
                          )

        self.root.after(FRAME_DELAY, self._tick)

    # ---- actions ----

    def _toggle_pause(self):
        self.paused = not self.paused
        self.play_btn.configure(text="Play" if self.paused else "Pause")

    def _reset(self):
        self.brightness.set(0.0)
        self.contrast.set(1.0)
        self._sync_labels()

    def _save(self):
        if self.saving:
            return
        out_path = filedialog.asksaveasfilename(
            defaultextension=".avi",
            filetypes=[("AVI", "*.avi"), ("MP4", "*.mp4")],
            initialfile="edited.avi")
        if not out_path:
            return

        self.saving = True
```

```
        self.save_error = None
        self.save_progress = 0.0
        self._pending_out = out_path
        self.save_btn.state(["disabled"])
        self.status.configure(text="Saving…")

        # snapshot the settings so later slider moves don't affect this export
        args = (out_path, self.contrast.get(), self.brightness.get())
        threading.Thread(target=self._save_worker, args=args, daemon=True).start()
        self._poll_save()

    def _save_worker(self, out_path, alpha, beta):
        """Runs off the UI thread. Re-reads the source at full resolution."""
        cap = writer = None
        try:
            cap = cv.VideoCapture(self.path)
            fps = cap.get(cv.CAP_PROP_FPS) or 25.0
            w = int(cap.get(cv.CAP_PROP_FRAME_WIDTH))
            h = int(cap.get(cv.CAP_PROP_FRAME_HEIGHT))
            total = int(cap.get(cv.CAP_PROP_FRAME_COUNT)) or 1

            ext = os.path.splitext(out_path)[1].lower()
            fourcc = cv.VideoWriter_fourcc(*("mp4v" if ext == ".mp4" else "XVID"))
            writer = cv.VideoWriter(out_path, fourcc, fps, (w, h))
            if not writer.isOpened():
                raise IOError("VideoWriter could not be opened")

            n = 0
            while True:
                ret, frame = cap.read()
                if not ret:
                    break
                writer.write(cv.convertScaleAbs(frame, alpha=alpha, beta=beta))
                n += 1
                self.save_progress = min(100.0, 100.0 * n / total)
        except Exception as exc:
            self.save_error = str(exc)
        finally:
            if writer is not None:
                writer.release()
            if cap is not None:
                cap.release()
            self.saving = False

    def _poll_save(self):
        """UI-thread poller: never touch widgets from the worker thread."""
        self.progress["value"] = self.save_progress
        if self.saving:
```

```
380             self.root.after(100, self._poll_save)
381         else:
382             self.progress["value"] = 0 if self.save_error else 100
383             self.save_btn.state(["!disabled"])
384             if self.save_error:
385                 self.status.configure(text="Save failed", foreground="#b00
                        ")
386                 messagebox.showerror("Save failed", self.save_error)
387             else:
388                 # the export becomes the left panel's source from now on,
389                 # and survives a restart through settings.json
390                 self.saved_path = self._pending_out
391                 self._open_saved()
392                 self.cap.set(cv.CAP_PROP_POS_FRAMES, 0)
393                 self._store_settings()
394                 self.status.configure(text="Saved", foreground="#070")
395
396     def close(self):
397         self._store_settings()
398         self.cap.release()
399         if self.saved_cap is not None:
400             self.saved_cap.release()
401
402
403 if __name__ == "__main__":
404     root = tk.Tk()
405     root.resizable(False, False)
406     app = VideoEditor(root, SOURCE)
407     root.protocol("WM_DELETE_WINDOW", lambda: (app.close(), root.destroy()
            ))
408     root.mainloop()
```

Listing A.1. Complete source code (`main.py`).